Parcial 02

▼ Parte 1 — FTPS

> 💡 Montar un **FTP seguro (FTPS)** donde el **Servidor 1** (UFW) es la **única puerta de entrada** y reenvía el tráfico al **Servidor 2** (vsftpd), que el cliente no puede ver directamente.

▼ 1. Firewall

▼ 1. Configuración inicial y reenvío ✅

> **Qué te pide este punto:** Configurar UFW en el Servidor 1 con política `deny` para el tráfico entrante y para el enrutado (*routed*), habilitar el reenvío IP y crear reglas DNAT hacia `192.168.50.2`. También hay que demostrar que el Servidor 2 no es alcanzable directamente desde el cliente.

1.1. Conceptualización

Política `incoming` vs `routed`:

- `incoming` aplica al tráfico dirigido al propio Servidor 1.
- `routed` aplica al tráfico que solo *pasa por* el Servidor 1 hacia otra máquina. Con `deny routed`, el Servidor 1 no reenvía nada que no hayas permitido con `ufw route allow` (puntos 2 y 3).

---

1. Crear el VagrantFile. En este caso se usa una IP pública desde la que se va a poder acceder desde el exterior → cambiar por el **código estudiantil de cada uno.**

```
Vagrant.configure("2") do |config|
  config.vm.box = "bento/ubuntu-22.04"
```

```
  # SERVIDOR 2: vsftpd
  config.vm.define "servidor2" do |srv2|
    srv2.vm.hostname = "srv2-2235060"
    srv2.vm.network "private_network", ip: "192.168.50.2", virtualbox__
intnet: "red-interna"
    srv2.vm.provider "virtualbox" do |vb|
      vb.name = "parcial2-servidor2"
      vb.memory = 1024
    end
  end

  # SERVIDOR 1: Firewall UFW (IP pública + red interna)
  config.vm.define "servidor1" do |srv1|
    srv1.vm.hostname = "srv1-2235060"

    # Interfaz PÚBLICA (bridge): IP fija (debe estar libre)
    srv1.vm.network "public_network", ip: "192.168.1.150"

    # Interfaz INTERNA: conecta con el Servidor 2
    srv1.vm.network "private_network", ip: "192.168.50.3", virtualbox__
intnet: "red-interna"

    srv1.vm.provider "virtualbox" do |vb|
      vb.name = "parcial2-servidor1"
      vb.memory = 1024
    end
  end
end
```

2. Levantar maquinas y entrar al servidor 1 → **vagrant ssh servidor1**
3. **Obtener la IP pública**: Ejecutar **ip -br a:**
   a. **eth1:** La IP pública → 192.168.1.150
   b. **eth2:** La IP privada (interna) → 192.168.50.3
4. **Habilitar UFW con políticas deny**

```
sudo ufw allow ssh
sudo ufw default deny incoming
sudo ufw default deny routed
sudo ufw default allow outgoing
sudo ufw enable
sudo ufw status verbose
```

5. **Habilitar el reenvío de IP: sudo vim /etc/ufw/sysctl.conf** y descomentar la línea #net/ipv4/ip_forward=1
6. **Agregar las reglas DNAT para redireccionamiento:**

```
sudo vim /etc/ufw/before.rules

------------- CONTENIDO -------------
# ===== NAT TABLE RULES (Parcial 2) =====
*nat
:PREROUTING ACCEPT [0:0]
:POSTROUTING ACCEPT [0:0]

# DNAT: canal de control FTP (21) -> Servidor 2
-A PREROUTING -d 192.168.1.150 -p tcp --dport 21 -j DNAT --to-destinati
on 192.168.50.2

# DNAT: rango de puertos pasivos (datos) -> Servidor 2
-A PREROUTING -d 192.168.1.150 -p tcp --dport 50000:50010 -j DNAT --to-
destination 192.168.50.2

# MASQUERADE: el Servidor 2 responde a través del Servidor 1
-A POSTROUTING -d 192.168.50.2 -j MASQUERADE

COMMIT
# ===== FIN NAT =====
```

7. **Recargar y verificar:** `iptables -t nat -L PREROUTING -n` debe mostrar tus dos reglas DNAT.

```
sudo ufw reload
sudo iptables -t nat -L PREROUTING -n
```

8. **Demostrar que el Servidor 2 NO es alcanzable desde el cliente (Windows)**

```
# El Servidor 2 NO debe responder
Test-NetConnection 192.168.50.2 -Port 21 # Windo
ws
nc -zv -w 3 192.168.50.2 21 # Linux

# El Servidor 1 SÍ debe responder (la única puer
ta de entrada)
ping 192.168.1.150
```

▼ 2. Permitir solo el tráfico estrictamente necesario ✅

Qué te pide este punto: Abrir únicamente tres cosas y nada más:

- En el Servidor 1, el puerto **22/tcp** (administración SSH).
- Hacia el Servidor 2, mediante reglas `ufw route`, el puerto **21/tcp** (canal de control FTP).
- Hacia el Servidor 2, el rango **50000:50010/tcp** (canal de datos pasivo).

---

1. **Permitir el canal de control hacia el Servidor 2 (Puerto 21):**

```
sudo ufw route allow proto tcp from any to 192.168.50.2 port 21
```

   - **route allow:** permite tráfico reenviado (que pasa por el Servidor 1)
   - **proto tcp:** solo TCP, que es la que usa el canal de control
   - **from any:** desde cualquier origen
   - **to 192.168.50.2 port 21:** hacia el servidor 2, puerto 21

2. **Permitir el rango pasivo de datos hacia el Servidor 2 (50000:50010)**

```
sudo ufw route allow proto tcp from any to 192.168.50.2 port 50000:50010
```

3. **Recargar y verificar**

```
sudo ufw reload
sudo ufw status verbose
```

▼ 3. Demostrar el control de acceso ✅

- **Qué te pide este punto:** Demostrar con evidencia que el firewall controla el acceso en cuatro pasos:
  1. Con UFW activo **sin** la regla de reenvío del puerto 21, intentar conectarte a `<ip pública>` desde el cliente: **debe fallar**.
  2. Agregar la regla.
  3. Confirmar que la conexión ahora **sí funciona**.

4. Explicar la diferencia entre `ufw allow` y `ufw route allow`.
    - **ufw allow:** Aceptar tráfico que va dirigido al propio servidor
    - **ufw route allow:** Aceptar tráfico que pasa por el servidor pero va hacia otro.

---

1. **Instalar vsftpd** en el SERVIDOR 2

```
sudo apt update && sudo apt install -y vsftpd
```

2. Eliminar temporalmente la regla de reenvío del puerto 21 (SERVIDOR 1)

```
sudo ufw route delete allow proto tcp from any t
o 192.168.50.2 port 21
sudo ufw reload
```

3. Probar desde Windows → **ftp 192.168.1.150** Debe quedarse esperando y terminar con un error de conexión (*timeout*).
4. **Agregar la regla de nuevo** (Servidor 1)

```
sudo ufw route allow proto tcp from any to 192.1
68.50.2 port 21
sudo ufw reload
```

5. Reintentar conexión de FTP desde Windows → **ftp 192.168.1.150**, como resultado debería de poder conectarse

▼ 4. Mostrar el estado del firewall y las reglas NAT ✅

- **Qué te pide este punto:** Mostrar la salida de dos comandos y explicar **qué hace cada regla dentro del servicio FTPS**:

1. Mostrar el estado de UFW → `sudo ufw status verbose`
2. Mostrar las reglas NAT activas → **sudo iptables -t nat -L -n -v**

▼ 2. Servicio FTPS

▼ 5. vsftpd en modo FTPS con TLS explícito ✅

- **Qué te pide este punto:** Instalar vsftpd en el Servidor 2 con TLS explícito, obligar el cifrado del login y de los datos (`force_local_logins_ssl=YES` y `force_local_data_ssl=YES`) y deshabilitar SSLv2/SSLv3. Para eso se usa un certificado y una clave privada.
- **Por qué lo pide el profesor:** FTP plano (por defecto) envía usuario, contraseña y archivos en texto claro → **no hay cifrado**.

5.1. Conceptualización

- **FTPS explícito:** La conexión empieza por el puerto 21 y el cliente envía `AUTH TLS` para **subir a TLS**.
- **CA propia:** Una CA es la entidad que **firma** certificados. Creamos una CA (`ca.crt`, `ca.key`) y con ella firmamos el certificado del servidor (`servidor.crt`)
- **CSR:** Es la solicitud "*esta es mi clave pública y mi identidad, firmamela*" (Solicitud de Firma)
- **SAN (Subject Alternative Name):** Lista de nombres/IPs válidos del certificado. Como el cliente se conecta por IP, el certificado incluye `IP:192.168.1.150`.
- **SSLv2/SSLv3:** Protocolos obsoletos y vulnerables. Se desactivan para que solo se negocie TLS.

---

1. **Crear la CA, el certificado y la clave**

```
# Certificado y clave se guardan en /etc/ssl/ftp
s
sudo mkdir -p /etc/ssl/ftps && cd /etc/ssl/ftps

# CA: clave privada y certificado autofirmado
sudo openssl genrsa -out ca.key 4096
sudo openssl req -x509 -new -nodes -key ca.key -
sha256 -days 3650 -out ca.crt \
  -subj "/C=CO/ST=Valle del Cauca/L=Cali/O=UAO/O
```

```
U=Servicios Telematicos/CN=CA-UAO"

# Generar la clave privada del servidor y solici
tud de firma (CSR) para la CA
sudo openssl genrsa -out servidor.key 2048
sudo openssl req -new -key servidor.key -out ser
vidor.csr \
  -subj "/C=CO/ST=Valle del Cauca/L=Cali/O=UAO/O
U=Servicios Telematicos/CN=192.168.1.150"

# Extensión SAN con la IP pública, y firma del c
ertificado con la CA
echo "subjectAltName=IP:192.168.1.150" | sudo te
e servidor.ext
sudo openssl x509 -req -in servidor.csr -CA ca.c
rt -CAkey ca.key -CAcreateserial \
  -out servidor.crt -days 365 -sha256 -extfile s
ervidor.ext

# Verificar que todo se creó correctamente
ls -l
```

2. Crear un usuario por el cual nos autenticaremos con FTP → **sudo adduser ftpuser**
3. Configurar **vsftpd.conf**

```
sudo vim /etc/vsftpd.conf

# Reemplazar todo el contenido por:
listen=YES
listen_ipv6=NO
anonymous_enable=NO
local_enable=YES
write_enable=YES
local_umask=022
dirmessage_enable=YES
use_localtime=YES
xferlog_enable=YES
connect_from_port_20=YES
secure_chroot_dir=/var/run/vsftpd/empty
pam_service_name=vsftpd

# ---- FTPS (TLS explícito) ----
```

```
ssl_enable=YES
rsa_cert_file=/etc/ssl/ftps/servidor.crt
rsa_private_key_file=/etc/ssl/ftps/servidor.key
force_local_logins_ssl=YES
force_local_data_ssl=YES
ssl_sslv2=NO
ssl_sslv3=NO
ssl_ciphers=HIGH
require_ssl_reuse=NO
```

- ssl_enable=YES: Activa TLS en vsftpd
- rsa_cert_file / rsa_private_key_file: Certificado y clave privada del servidor
- force_local_logins_ssl=YES: Rechaza el login si no se hizo antes AUTH TLS
- force_local_data_ssl=YES: Rechaza transferencia de datos sin cifrar
- ssl_sslv2=NO / ssl_sslv3=NO: Deshabilita protocolos obsoletos
- ssl_ciphers=HIGH: Solo suites de cifrado fuertes
- require_ssl_reuse=NO: Evita fallos de FileZilla

4. Reiniciar vsftpd → **sudo service vsftpd restart**

▼ 6. Rango pasivo y pasv_address ✅

- **Qué te pide este punto:** Configurar en vsftpd `pasv_min_port=50000`, `pasv_max_port=50010` y `pasv_address=<ip pública>`, de forma consistente con las reglas de UFW. Además, explicar (a), (b) y (c).
- **Modo pasivo:** El cliente pide `PASV` y el servidor responde con **una IP y un puerto** (dentro del rango) donde esperará la conexión de datos.
- `pasv_address`: La IP que vsftpd anuncia al cliente para conectarse.

---

1. Agregar las directivas de vsftpd y reiniciar

```
sudo vim /etc/vsftpd.conf

# Agregar el contenido:
# ---- Modo pasivo ----
```

```
pasv_min_port=50000
pasv_max_port=50010
pasv_address=192.168.1.150

sudo service vsftpd restart
```

2. Responder a las preguntas:
    a. **¿Por qué el rango pasivo debe coincidir con las reglas del firewall?:** Porque el firewall solo deja pasar el rango 50000:50010. Si vsftpd anunciara un puerto fuera de ese rango, **el paquete se descartaría** por la política deny routed.
    b. **¿Por qué en FTPS el firewall no puede abrir puertos dinámicamente?:** En FTP plano, el firewall lee la respuesta PASV (porque **viaja en texto plano**) y abre ese puerto temporalmente. En FTPS ya no puede hacer eso porque viene cifrado, por lo que hay que abrir un rango fijo.
    c. **¿Qué pasa si pasv_address no se configura destrás de NAT?:** vsftpd anuncia su propia IP interna. El cliente intenta conectarse a esa dirección privada que no puede alcanzar (porque es privada) → timeout

▼ 7. Rango pasivo y pasv_address ✅

- **Qué te pide este punto:** Conectarte con FileZilla en modo **FTP explícito sobre TLS** a `<ip pública>`, analizar el certificado que presenta el servidor (sujeto, emisor, vigencia, huella) y compararlo con la huella que calcula OpenSSL. Luego listar el directorio remoto, subir `<codigo_estudiantil>.txt` y descargarlo de vuelta.
- **Huella (fingerprint):** Es el hash SHA-256 del certificado completo. Si coincide en el cliente y en el servidor, es el mismo certificado.

---

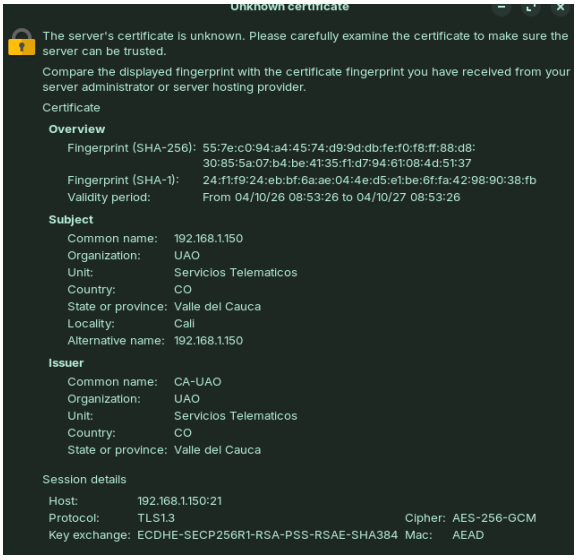
1. **Obtener la huella den el servidor 2:**

```
sudo openssl x509 -in /etc/ssl/ftps/servidor.crt
-noout -fingerprint -sha256
```

2. Crear el **archivo a subir** en Windows (en la misma carpeta Parcial 02)

```
echo "Prueba FTPS <codigo_estudiantil>" > <codigo_estudiantil>.txt
```

3. Configurar la **conexión en FileZilla:** Abre **Archivo** → **Gestor de sitios** → **Nuevo sitio** y completa:
   - Protocolo: FTP - File Transfer Protocol
   - Servidor: 192.168.1.150
   - Puerto: 21
   - Cifrado: **Requerir FTP explícito sobre TLS**
   - Tipo de acceso: Ask for Password
   - Usuario: ftpuser
4. **Analizar el certificado:** Presionar el botón **Conectar en el Site**. Luego presionar OK



- **Sujeto:** A quién pertenece el certificado
- **Emisor:** Quién lo firmó (CA)
- **Huella:** Identidad del certificado (Igual a la del paso 1)

5. **Listar, subir y descargar**
   a. Una vez conectado se ve el listado delhome de ftpuser
   b. **Subir:** Subir el archivo **<codigo_estudiantil>.txt** → **sudo ls /home/ftpuser**
   c. **Descargar:** Arrastrar el archivo remoto a otra carpeta local

▼ 8. Validar el certificado con openssl s_client ✅

- **Qué te pide este punto:** Desde el cliente, ejecutar **openssl s_client** contra la ip publica en el puerto 21 para iniciar FTP con el certificado y confirmar 3 cosas:
  - El certificado corresponde al servidor configurado
  - La cadena de confianza es válida (Verify return code: 0 (ok))
  - Cuál es la versión de TLS y la suite de cifrado negociadas.
- `s_client`: Cliente TLS de OpenSSL.
- `starttls ftp`: FTPS explícito.
- `CAfile ca.crt`: CA en la que el cliente confía. OpenSSL comprueba que el certificado del servidor esté firmado por ella.
- **Suite de cifrado:** Conjunto de algoritmos que protegen la sesión (intercambio de claves, cifrado y autenticación).

---

1. Pasar el ca.crt de servidor 2 a Windows con directorios sincronizados

```
sudo cp ca.crt /vagrant/
```

2. Ejecutar la conexión desde Windows y obtener la salida importante

```
openssl s_client -connect 192.168.1.150:21 -starttls ftp -CAfile ca.crt -verify_ip 192.168.1.150 </dev/null 2>&1 | grep -E "^ *0 s:|^ *i:|Verification|New,|Verify return"
```

3. Ubicar cada evidencia en la salida

Qué buscar	Qué debe mostrar	Qué demuestra
`s:`	`CN = 192.168.1.150`	El certificado es del servidor configurado
`i:`	`CN = CA-UAO`	Lo firmó tu CA
`Verification:`	`OK`	La firma es válida

Qué buscar	Qué debe mostrar	Qué demuestra
`Verify return code:`	`0 (ok)`	La cadena de confianza es válida
`Protocolo y Cipher`	New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384	Versión de TLS y suite de cifrado negociada

▼ 9. Capturas con Wireshark ✅

- **Qué te pide este punto:** Capturar dos sesiones y conservar ambas capturas (se reutilizan en la Parte 3):
  1. **FTP sin cifrar** (con TLS deshabilitado temporalmente): identificar `USER` / `PASS` y el contenido del archivo en texto plano.
  2. **FTPS**: identificar el comando `AUTH TLS`, el handshake TLS en el puerto 21 y el tráfico cifrado en los puertos pasivos.

---

1. Preparar la **sesión sin cifrar**

```
# Guardar la configuración FTPS actual
sudo cp /etc/vsftpd.conf /etc/vsftpd.conf.ftps

# Deshabilitar TLS temporalmente
sudo sed -i 's/^ssl_enable=YES/ssl_enable=NO/;
s/^force_local_logins_ssl=YES/force_local_logins
_ssl=NO/; s/^force_local_data_ssl=YES/force_loca
l_data_ssl=NO/' /etc/vsftpd.conf
sudo service vsftpd restart
```

   - `sed -i 's/A/B/'`: cambia `YES` por `NO` en esas tres directivas, directamente en el archivo.

2. **Capturar la sesión FTP sin cifrar**
   a. Abre Wireshark y elige el adaptador que usa tu PC para la red (Wi-Fi o Ethernet)
   b. En el campo de **filtro de captura** escribe **ftp**
   c. En FileZilla, crea o edita el sitio con **Cifrado: "Solo usar FTP simple (inseguro)"** (usuario `ftpuser`) y conecta.

d. Lista el directorio, **sube** un archivo y **descárgalo**.

e. Desconecta, detén la captura y guárdala como `ftp_plano.pcapng`.

f. **Ver contenido del archivo**

i. Cambiar el filtro a **ftp-data**

ii. Seleccionar un paquete que tenga datos → Click derecho → Seguir → Flujo TCP

3. Volver a **FTPS**:

```
sudo cp /etc/vsftpd.conf.ftps /etc/vsftpd.conf
sudo systemctl restart vsftpd
```

4. **Capturar la sesión FTPS**

a. Nueva captura en Wireshark con el mismo filtro **ftp**

b. En FileZilla, cambia el cifrado a **"Requerir FTP explícito sobre TLS"** y conecta (acepta el certificado).

c. Lista, sube el mismo archivo y descárgalo.

d. Desconecta, detén la captura y guárdala como `ftps.pcapng`.

e. **Analizar la captura**

Qué buscar	Filtro	Qué verás
`AUTH TLS`	`ftp`	`Request: AUTH TLS` y `Response: 234 Proceed with negotiation`.
Handshake TLS en el 21	`tcp.port == 21 && tls`	`Client Hello`, `Server Hello` y luego solo `Application Data`
Tráfico cifrado en los puertos pasivos	tcp.srcport >= 50000 && tcp.srcport <= 50010	Handshake TLS y `Application Data`. **Seguir flujo TCP** muestra caracteres ilegibles

▼

Parte 2 — DNS sobre TLS (DoT — Servidor 2)

> 💡 Cifrar las consultas DNS en un cliente Linux con **DNS sobre TLS (DoT)**, usando `systemd-resolved`. Normalmente el DNS viaja en claro por el puerto 53 UDP, y con DoT viaja dentro de TLS por el puerto 853 TCP.

▼ 10. Configurar resolved.conf para DoT ✅

- **Qué te pide este punto:** Editar `/etc/systemd/resolved.conf` con al menos dos resolvers públicos que soporten DoT (indicando el nombre del servidor), `DNSOverTLS=yes` y un `FallbackDNS=` coherente. Luego verificar que `/etc/resolv.conf` apunte al stub `127.0.0.53`, reiniciar el servicio y explicar la diferencia entre `yes` y `opportunistic`.

10.1. Conceptualización

- `systemd-resolved`: Servicio de Ubuntu que gestiona el DNS. Las aplicaciones consultan a un **stub local** (`127.0.0.53`) y este reenvía la consulta a los servidores configurados.
- **DoT:** Envuelve la consulta DNS en TLS sobre **TCP 853**.
- `IP#nombre` (por ejemplo `1.1.1.1#cloudflare-dns.com`): La IP es a dónde se conecta, y el nombre después del `#` es el que se usa para **validar el certificado** del servidor. Sin el nombre, `systemd-resolved` no puede comprobar que habla con quien dice ser.
- `FallbackDNS`: Servidores de respaldo que se usan solo si no hay otros configurados. Deben ser coherentes: mismos proveedores y también con DoT.

---

1. Verificar el **servicio y crear un respaldo** (copia de seguridad)

```
systemctl is-active systemd-resolved
sudo cp /etc/systemd/resolved.conf /etc/systemd/res
```

```
olved.conf.bak
```

2. **Editar resolved.conf:** Limpiar todo y escribir:

```
sudo vim /etc/systemd/resolved.conf

# Contenido
[Resolve]
DNS=1.1.1.1#cloudflare-dns.com 8.8.8.8#dns.google
FallbackDNS=1.0.0.1#cloudflare-dns.com 8.8.4.4#dns.google
DNSOverTLS=yes
Domains=~.
```

   - **DNS:** Servidores principales
   - **FallbackDNS:** Resplado de los mismos proveedores (otras IPs)
   - **DNSOverTLS:** Obliga a cifrar todas las consultas
   - **Domains:** Todas las consultas van primero a los servidores globales, no al DNS sin DoT.

3. Verificar el stub (que sea nameserver 127.0.0.53) y reiniciar systemd-resolved

```
sudo vim /etc/resolv.conf
sudo systemctl restart systemd-resolved
resolvectl query example.com # Query de ejemplo
```

4. **Respuesta sustentación:**

	`DNSOverTLS=yes` (estricto)	`DNSOverTLS=opportunistic`
Comportamiento	Todas las consultas van por TLS	Intenta TLS, y si el servidor no lo soporta, **cae a DNS plano**
Validación del certificado	Sí (con el nombre tras `#`)	No
Si el 853 está bloqueado	El DNS **deja de funcionar**	Sigue funcionando en texto claro
Riesgo	Disponibilidad	Un atacante puede forzar el *downgrade* a texto plano

▼ 11. Verificar que DoT está activo con resolvectl status ✅

- **Qué te pide este punto:** Ejecutar `resolvectl status`, mostrar la salida completa y señalar **dónde se ve que DoT está activo** y que los servidores configurados son los correctos.

---

1. **Mostrar el estado completo: resolvectl status**
2. **Buscar la evidencia:**

Qué demostrar	Dónde mirar	Qué debe decir
DoT activo	Sección `Global`, línea `Protocols`	`+DNSOverTLS`
Servidores correctos	`DNS Servers`	`1.1.1.1#cloudflare-dns.com 8.8.8.8#dns.google`
Nombre para validar el certificado	Texto tras el `#`	`cloudflare-dns.com` y `dns.google`
Respaldo coherente	`Fallback DNS Servers`	`1.0.0.1#...` y `8.8.4.4#...`
Pasa por el stub local	`resolv.conf mode`	`stub` (`127.0.0.53`)
Todas las consultas van a los globales	`DNS Domain`	`~.`

▼ 12. Resolver dominios a través de systemd-resolved ✅

- **Qué te pide este punto:** Resolver al menos **tres dominios distintos** con `resolvectl query` o `dig` **sin** `@servidor`, para que la consulta pase por `systemd-resolved`. Además, explicar por qué `dig @8.8.8.8 <dominio>` no usaría DoT.

---

1. **Resolver tres dominios con resolvectl**

```
resolvectl flush-caches # Limpiar cache
resolvectl query google.com
resolvectl query uao.edu.co
resolvectl query youtube.com
```

> *Verificar: Data was acquired via local or encrypted transport: yes*

2. Probar con dig sin @

```
sudo apt install -y dnsutils
dig github.com +short
dig github.com | grep SERVER # Pasa por el STUB
```

3. Comparar con una **consulta directa a 8.8.8.8 con @**
   - `dig @IP`**:** Ignora `/etc/resolv.conf` y envía la consulta **directamente** a esa IP.
   - Debe mostrar `;; SERVER: 8.8.8.8#53(8.8.8.8)`: la consulta salió **directo al puerto 53, en texto plano**, sin pasar por el stub.

```
dig @8.8.8.8 github.com | grep SERVER
```

▼ 13. Capturar DNS con DoT activo y sin cifrar ✅

- **Qué te pide este punto:** Con la caché limpia, capturar una consulta con **DoT activo** (filtro `tcp.port == 853`) y confirmar que viaja cifrada. Luego deshabilitar DoT (`DNSOverTLS=no`), capturar una consulta convencional (filtro `udp.port == 53`), mostrar ambas capturas y explicar **qué información queda expuesta** en el DNS sin cifrar.
- **Dónde capturar:** El Servidor 2 es una VM sin interfaz gráfica, así que se captura con `tcpdump`, se guarda el `.pcap` en `/vagrant` (carpeta compartida) y se abre en **Wireshark en Windows**.

> 💡 **SE NECESITAN DOS TERMINALES CONECTADAS AL SERVIDOR 2**

1. **Captura con DoT activo**
   a. **Terminal 1:** Iniciar la captura

```
sudo tcpdump -Z root -i eth0 -nn 'tcp port 853'
-w /vagrant/dot.pcap
```

   - **-Z root:** Evita errores de permisos al escribir en la carpeta compartida

- **-nn:** No resuelve nombres ni puertos
- **'tcp port 853':** Solo captura el tráfico DoT
- **-w:** Guarda en un archivo (carpeta compartida)

b. **Terminal 2:** Genera la consulta

```
sudo systemctl restart systemd-resolved
resolvectl flush-caches
resolvectl query uao.edu.co
```

c. Vuelve a la **Terminal 1** y detén con Control + C. Debe indicar X packets captured

2. **Deshabilitar DoT temporalmente**

```
sudo sed -i 's/^DNSOverTLS=yes/DNSOverTLS=no/' /et
c/systemd/resolved.conf
sudo systemctl restart systemd-resolved
```

3. **Captura con DNS convencional**

a. **Terminal 1:**

```
sudo tcpdump -Z root -i eth0 -nn 'udp port 53' -
w /vagrant/dns_plano.pcap
```

b. **Terminal 2:**

```
resolvectl flush-caches
resolvectl query uao.edu.co
```

c. Detener la captura en la terminal 1

4. **Restaurar DoT**

```
sudo sed -i 's/^DNSOverTLS=no/DNSOverTLS=yes/' /et
c/systemd/resolved.conf
sudo systemctl restart systemd-resolved
```

5. **Analizar en Wireshark**

- Abrir dot.pcap y dns_plano.pcap desde la carpeta Parcial 2 en Wiresahrk

`dot.pcap:`

Qué buscar	Qué verás
Handshake TCP	`SYN`, `SYN/ACK`, `ACK` hacia `1.1.1.1:853`
Handshake TLS	`Client Hello` y `Server Hello`
Consulta y respuesta	Solo `Application Data`
Contenido	Clic derecho → **Seguir** → **Flujo TCP**: caracteres ilegibles

- No aparece ningún paquete DNS decodificado: el dominio consultado no se puede leer.

`dns_plano.pcap`

Qué buscar	Qué verás
Consulta	`Standard query 0x... A uao.edu.co`
Respuesta	`Standard query response ... A <IP>`
Detalle	Despliega `Domain Name System` y verás `Queries` (dominio y tipo) y `Answers` (IPs)

6. **Respuesta para la sustentación: ¿Qué queda expuesto en DNS sin cifrar?**
   - **Dominio consultado (QNAME):** cualquiera en el camino lee a qué sitios te conectas (`uao.edu.co`), aunque el sitio use HTTPS.
   - **Tipo de registro:** `A`, `AAAA`, `MX`, etc., que revela qué haces (web, correo...).
   - **Respuesta con las IPs:** a qué servidores vas a conectarte.
   - **IP de origen y del resolver:** quién consulta y a quién.

▼

14. Qué sigue siendo visible con DoT, bloqueo del 853 y DoT vs DoH ✅

- **Qué te pide este punto:** Responder por escrito tres preguntas:

1. Con DoT activo, ¿qué información sigue siendo visible para un observador de la red?
2. ¿Qué pasa si un firewall bloquea el puerto 853 con `DNSOverTLS=yes` frente a `opportunistic`?
3. Comparar brevemente DoT con DoH en este escenario.

Respuestas para la sustentación

1. ¿Qué sigue siendo visible con DoT?

Información	¿Visible?	Por qué
IP del resolver (`1.1.1.1`)	Sí	Es la IP de destino del paquete
Puerto 853	Sí	Delata que usas DoT
SNI del handshake (`cloudflare-dns.com`)	Sí	Viaja en el `Client Hello`, que va en claro (salvo con ECH)
Tamaños y tiempos de los paquetes	Sí	TLS no los oculta; permite deducir, por ejemplo, cuántas consultas haces y cuándo
IP y SNI de la **conexión posterior** al sitio	Sí	Después de resolver, te conectas al sitio y esa conexión revela su IP y su SNI

- Un observador no lee **qué** preguntas, pero sí ve **que** consultas DNS cifrado, a **quién** y **cuánto**.

2. ¿Qué pasa si se bloquea el puerto 853?

	`DNSOverTLS=yes`	`DNSOverTLS=opportunistic`
Resultado	La resolución **falla** (timeout o `SERVFAIL`): el sistema se queda sin DNS	La resolución **sigue funcionando**, pero en **texto plano** por el puerto 53
Privacidad	Se mantiene (no se filtra nada)	Se pierde sin avisar (*downgrade*)
Disponibilidad	Se pierde	Se mantiene
Riesgo	Denegación de servicio	Un atacante puede bloquear el 853 a propósito para forzar el DNS en claro

- `yes` prioriza la **seguridad**, y `opportunistic` la **disponibilidad**.

3. DoT vs DoH en este escenario

	DoT	DoH
Transporte	DNS sobre TLS	DNS dentro de peticiones HTTPS
Puerto	**853/TCP** dedicado	**443/TCP**, el mismo de la web
Identificación	Fácil de reconocer y filtrar por puerto	Se mezcla con el tráfico web normal
Si un firewall bloquea el 853	Deja de funcionar	**Sigue funcionando**, porque el 443 no se bloquea
Control del administrador	Más fácil de monitorear o bloquear	Más difícil de controlar
Soporte en `systemd-resolved`	Sí	No

- **En este escenario:** si el firewall bloquea el 853, DoT cae y DoH sigue disponible.
- **Lo que ambos comparten:** ocultan las consultas a quien observa la red, pero **el resolver sí las ve**, y ambos dejan visibles IP y SNI.

▼ Parte 3 — SFTP

> 💡 Montar una alternativa a FTPS con **SFTP sobre OpenSSH** en el Servidor 2. A diferencia de FTPS (varias conexiones y TLS), SFTP usa **un único canal SSH cifrado** para autenticación y datos.

- **Cómo se accede:** Solo a través del firewall. Para no chocar con el SSH de administración del Servidor 1 (22), se publica el **puerto 2222 del Servidor 1**, reenviado al **22 del Servidor 2**.

▼ 15. Usuario exclusivo para SFTP con chroot ✅

- **Qué te pide este punto:** Configurar OpenSSH en el Servidor 2 con un usuario `sftp_<codigo_estudiantil>` restringido a su directorio mediante `Match User`, `ChrootDirectory` y `ForceCommand internal-sftp`. Debes demostrar que **puede usar SFTP pero no abrir una shell**.

15.1. Conceptualización

- **SFTP:** Es un subsistema de SSH (no tiene relación con FTP): corre por el puerto 22 y hereda su cifrado.
- `Match User`: Aplica las directivas siguientes solo a ese usuario. Debe ir **al final** de `sshd_config`.
- `ChrootDirectory`: Encierra al usuario en una carpeta, que para él es la raíz `/`. OpenSSH exige que esa carpeta sea de **root y sin escritura para otros**, por eso el usuario escribe en la subcarpeta `upload`.
- `ForceCommand internal-sftp`: Obliga a ejecutar solo el servidor SFTP interno, ignorando cualquier comando o shell que pida el cliente.

---

1. **Crear el usuario y su jaula (Servidor 2)**

```
# Crear una variable de entorno
U=sftp_<codigo_estudiantil>

# Crea el usuario con una shell inválida y sin carp
eta personal
sudo adduser --shell /usr/sbin/nologin --home /srv/
sftp/$U --no-create-home $U

sudo mkdir -p /srv/sftp/$U/upload

# La jaula debe ser root:root con 755 o OpenSSH rec
haza la conexión
sudo chown root:root /srv/sftp /srv/sftp/$U
sudo chmod 755 /srv/sftp /srv/sftp/$U

# Se asigna como propietario al usuario en la carpe
ta para subidas
sudo chown $U:$U /srv/sftp/$U/upload

# Verificar que todo se creó bien
ls -ld /srv/sftp /srv/sftp/$U /srv/sftp/$U/upload
```

2. **Configurar sshd_config (Servidor 2)**

```
# Copia de respaldo
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak
sudo vim /etc/ssh/sshd_config

# Dejar el final del archivo como
UseDNS no
GSSAPIAuthentication no

Match User sftp_<codigo_estudiantil>
    # Raíz de la jaula del usuario
    ChrootDirectory /srv/sftp/%u
    # Solo SFTP, nunca una shell
    ForceCommand internal-sftp
    # Sin terminal
    PermitTTY no
    # Sin túneles
    AllowTcpForwarding no
    X11Forwarding no

#-----
sudo sshd -t # Revisar sintaxis
sudo systemctl restart ssh

# Muestra la configuración efectiva de ese usuario
sudo sshd -T -C user=$U,host=localhost,addr=127.0.0.1 | grep -E "chrootdire
ctory|forcecommand|permittty"
```

3. **Configurar el renvío 2222 → 22 (Servidor 1)**

```
# Agregar regla en el before.rules
sudo sed -i '/--dport 50000:50010/a -A PREROUTING -
d 192.168.1.150 -p tcp --dport 2222 -j DNAT --to-de
stination 192.168.50.2:22' /etc/ufw/before.rules

# Agregar regla de renvío con ufw route allow
sudo ufw route allow proto tcp from any to 192.168.
50.2 port 22
sudo ufw reload
sudo ufw status verbose
sudo iptables -t nat -L PREROUTING -n | grep 2222
```

4. Demostrar que **SFTP funciona** (desde Windows)

```
sftp -P 2222 sftp_<codigo_estudiantil>@192.168.1.15
0
```

```
# Demostrar que no se puede abrir una shell
ssh -p 2222 sftp_<codigo_estudiantil>@192.168.1.150
```

▼ 16. Reenvío 2222 → 22 y demostración del bloqueo ✅

- **Qué te pide este punto:** En el servidor 1, agregar el reenvío 2222 → 192.168.50.2:22 y la regla ufw route correspondiente. Debes demostrar que:
  - Sin la regla, la transferencia SFTP falla, y al habilitarla es exitosa
  - El puerto 22 del servidor no es accesible directamente desde el cliente.

---

1. Quitar la regla de reenvío en el servidor 1 momentáneamente

```
sudo ufw route delete allow proto tcp from any to 1
92.168.50.2 port 22
sudo ufw reload
sudo ufw status verbose
```

2. Probar desde Windows: **DEBE FALLAR**

```
sftp -P 2222 -o ConnectTimeout=5 sftp_<codigo_estud
iantil>@192.168.1.150
```

3. Agregar la regla de nuevo

```
sudo ufw route allow proto tcp from any to 192.168.
50.2 port 22
sudo ufw reload
sudo ufw status verbose
```

4. Verificar que el puerto 22 del Servidor 2 NO es accesible directamente

```
ssh -o ConnectTimeout=5 sftp_<codigo_estudiantil>@1
92.168.50.2
```

▼

17. Conectar por SFTP, verificar la huella del host y transferir ✅

- **Qué te pide este punto:** Desde el cliente, conectarte con `sftp -P 2222 sftp_<codigo_estudiantil>@<ip pública>`, verificar la **huella de la clave de host** que se muestra en la primera conexión y documentar tres operaciones: `ls`, `put` y `get`.
- **Clave de host SSH:** Cada servidor SSH tiene su propio par de claves (`/etc/ssh/ssh_host_*`). En la primera conexión, el cliente muestra la huella de la clave pública del servidor y pregunta si confías en ella.
- **TOFU (*Trust On First Use*):** Si aceptas, la clave se guarda en `~/.ssh/known_hosts`. En las siguientes conexiones, SSH la compara, y si cambia, **avisa de un posible ataque** (*man-in-the-middle*).

---

1. Obtener la huella real del Serviodr 2

```
for f in /etc/ssh/ssh_host_*_key.pub; do echo "$f";
ssh-keygen -lf $f; done
```

   - Copiar la clave de **ED25519**; es la que suele negociar el cliente.

2. **Limpiar registros anteriores (Windows):** Si ya te conectaste antes (punto 15 o 16), no aparecerá la pregunta de la primera conexión. Bórrala para volver a verla.

```
ssh-keygen -R "[192.168.1.150]:2222"
```

3. **Conectarse y verificar la huella (Windows)**

```
cd "ruta\a\Parcial 2"
echo "prueba sftp" > prueba.txt
sftp -P 2222 sftp_<codigo_estudiantil>@192.168.1.15
0
```

   - **Compara** esa huella con la del paso 1 (mismo tipo de clave).

4. **Listar, subir y descargar (dentro de sftp)**

```
pwd
ls
cd upload
put prueba.txt
ls -l
get prueba.txt descargado.txt
lls # Lista la carpeta localmente
bye
```

▼ 18. Capturar SFTP y compararlo con FTP y FTPS ✅

- **Qué te pide este punto:** Capturar una sesión SFTP con el filtro `tcp.port == 2222` e identificar el intercambio de versiones (`SSH-2.0`) y el intercambio de claves. Debes demostrar que **autenticación y datos viajan cifrados en un único canal**, y comparar con las capturas FTP y FTPS de la Parte 1 (número de conexiones, puertos y qué es visible).
- **Orden de una sesión SSH:** El cliente y el servidor intercambian sus **versiones** en claro (`SSH-2.0-...`), negocian algoritmos (`Key Exchange Init`), acuerdan una clave de sesión (ECDH) y envían `New Keys`. **Desde ese punto todo va cifrado**: login, comandos SFTP y archivos.

---

1. **Capturar la sesión:** Abrir Wireshark, elegir el adaptador de red Wifi o Ethernet y poner en los filtros **tcp.port == 2222**
2. Conectarse a SFTP

```
sftp -P 2222 sftp_<codigo_estudiantil>@192.168.1.15
0
```

3. Dentro de sftp

```
cd upload
put prueba.txt
ls -l
get prueba.txt descargado.txt
bye
```

4. Detener la captura y guardarla como **sftp.pcapng**
    a. Para que la **información pueda ser leída:**
        i. Seleccionar cualquier paquete de la captura
        ii. Analyze → Decode As...
        iii. Añadir una fila con estos valores
            1. **Field:** TCP port
            2. **Value:** 2222
            3. **Current:** SSH
5. **Identificar cada fase21**

Fase	Qué aparece en `Info`	Qué demuestra
Conexión TCP	`[SYN]` → `[SYN, ACK]` → `[ACK]`	Se establece la conexión TCP con el puerto **2222**.
Versiones	`Client: Protocol (SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.19)` / `Server: Protocol (SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.13)`	El cliente y servidor intercambian sus versiones de SSH **en claro**.
Negociación	`Client: Key Exchange Init` / `Server: Key Exchange Init`	Intercambian las propuestas de algoritmos que utilizarán para establecer la sesión.
Intercambio de claves	`Client: Diffie-Hellman Key Exchange Init` / `Server: Diffie-Hellman Key Exchange Reply, New Keys`	Se realiza el intercambio de claves mediante **Diffie-Hellman** y el servidor indica `New Keys`.
Fin de la negociación	`Client: New Keys`	El cliente confirma el cambio a las nuevas claves. Desde este punto, el contenido posterior de SSH está cifrado.
Autenticación y datos	`SSHv2 Client:` / `SSHv2 Server:` sin mostrar el contenido de las operaciones	Se intercambian los mensajes de autenticación y posteriormente los datos de la sesión, pero **Wireshark ya no muestra su contenido en claro**.

Fase	Qué aparece en `Info`	Qué demuestra
Cierre	`[FIN, ACK]` → `[ACK]` → `[FIN, ACK]` → `[ACK]`	Se cierra correctamente la conexión TCP.

6. **Comparación con las capturas de la Parte 1**

	FTP plano	**FTPS**	**SFTP**
Conexiones	1 de control + 1 **por operación** de datos	Igual: 1 de control + 1 por operación (con TLS en cada una)	**1 sola**
Puertos	21 + rango pasivo	21 + rango pasivo	**2222** (22 en el Servidor 2) → ÚNICO PUERTO
Usuario y contraseña	Visibles (`USER` , `PASS`)	Cifrados	Cifrados
Comandos (`STOR` , `RETR` , `LIST`)	Visibles	Cifrados	Cifrados
Contenido de archivos	Legible	Cifrado	Cifrado
Visible en claro	Todo	`AUTH TLS` , `Client Hello` y `Server Hello`	Banners `SSH-2.0` , `Key Exchange Init, etc.`
Cuándo empieza el cifrado	Nunca	Tras `AUTH TLS` (el inicio va en claro)	Tras `New Keys` , al terminar la negociación SSH

▼ 19. Tabla comparativa FTPS vs SFTP y Conclusión ✅

- **Qué te pide este punto:** Una tabla comparativa con seis criterios (protocolo base, conexiones y puertos, autenticación del servidor, inicio del cifrado, facilidad para atravesar firewalls/NAT y facilidad de configuración), y una conclusión **basada en tus propias evidencias**: cuál es más adecuado con restricciones de firewall estrictas y por qué.

---

1. **Tabla comparativa**

Criterio	**FTPS**	**SFTP**
Protocolo base	FTP + TLS (vsftpd)	Subsistema de SSH (OpenSSH). **No** es FTP
Conexiones y puertos	Control en el 21 + **una conexión de datos por operación** en el rango pasivo (50000:50010)	**Una sola conexión** TCP (22, publicado como 2222)
Autenticación del servidor	**Certificado X.509** firmado por una CA. El cliente valida la cadena de confianza	**Clave de host SSH**. El cliente valida la huella (TOFU y `known_hosts`)
Cuándo inicia el cifrado	Tras `AUTH TLS`: el inicio del canal de control va en claro. Cada canal de datos negocia su propio TLS	Tras el intercambio de claves (`New Keys`). Solo la negociación va en claro
Firewall y NAT	**Difícil**: el canal de control cifrado impide al firewall leer el `PASV`, así que exige rango fijo, `pasv_address` y reglas en tres capas	**Fácil**: un solo puerto, sin puertos dinámicos ni `pasv_address`
Facilidad de configuración	**Mayor esfuerzo**: CA, clave, CSR, certificado firmado, directivas TLS y pasivas en vsftpd, y reglas NAT y `route`	**Menor esfuerzo**: OpenSSH ya está instalado, y basta crear el usuario, el chroot y un bloque `Match`

2. Conclusiones finales

   SFTP es más adecuado para un entorno con restricciones de firewall estrictas. Lo que tú mismo configuraste y capturaste lo demuestra:

   - **Reglas necesarias (puntos 4 y 16):** FTPS requirió **dos DNAT y dos reglas** `route` (puerto 21 y rango 50000:50010). SFTP requirió **un DNAT y una regla** `route` (2222 → 22).
   - **Puertos expuestos:** FTPS abre **12 puertos** (21 + 11 pasivos). SFTP abre **1**.
   - **Capturas (puntos 9 y 18):** una transferencia FTP/FTPS abrió una conexión de datos por operación (en tu captura, los puertos 50003, 50007 y 50009 para `STOR`, `RETR` y `LIST`). En SFTP **toda** la sesión viajó por una única conexión al puerto 2222.

- **Dependencia de configuración (punto 6):** FTPS solo funciona si el rango pasivo coincide en vsftpd, DNAT y UFW, y si `pasv_address` está bien. SFTP no tiene ese punto de fallo.
- **Por qué el firewall no puede ayudar en FTPS:** con el canal de control cifrado, no puede leer el puerto que anuncia el servidor, así que no puede abrir puertos dinámicamente.